

errorHandlingPlan

Error Review Feature — Implementation Plan

Context

TEP Uploader uploads XML files to S3 (incoming/). A remote system processes them asynchronously, moving files to complete/ on success or errors/ on failure (with a companion .json error report). Users need to see which uploads succeeded, which failed, and what went wrong — then dismiss batches once dealt with. There's already a stub ErrorReview component and tab wired up.

Two linked changes: (1) add a batch ID to upload filenames so files can be grouped by upload session, and (2) build the full Error Review section.

Phase 1: S3 layer changes

src/lib/s3.js

- **Add import:** `GetObjectCommand` (from `@aws-sdk/client-s3`)
- **Modify** `makeS3Key(hash)` → `makeS3Key(hash, batchId)`: insert `{batchId}-` between hash and date segments. New format: `incoming/{hash}-{batchId}-{YYYYMMDD}-{HHMMSS}-{random6}.xml`
- **Add** `generateBatchId()`: returns 8-char hex string via `crypto.getRandomValues(new Uint8Array(4))`
- **Add** `TEP_FILENAME_REGEX`: `/^([a-f0-9]{64})-([a-f0-9]{8})-(\d{8})-(\d{6})-(\d{6})\.xml$/`
- **Add** `parseFilename(filename)`: returns `{ hash, batchId, date, time, random }` OR null
- **Add** `listAllObjects(s3, bucketName, prefix)`: paginated `ListObjectsV2Command` loop, returns flat array of all objects
- **Add** `downloadObject(s3, bucketName, key)`: `GetObjectCommand` → `result.Body.transformToString()`

src/steps/UploadProgress.svelte

- Import `generateBatchId` from `../lib/s3.js`
 - Generate `batchId` once at component top level: `const batchId = generateBatchId()`
 - Change line 100: `makeS3Key(hash)` → `makeS3Key(hash, batchId)`
-

Phase 2: Error cache module

New file: src/lib/errorCache.js

localStorage key: `tep_error_cache`. Follows the `load()/save()` pattern from `learnedAliases.js` (try/catch on parse, fall through to empty default).

Cache shape:

```
{
  batches: { "a1b2c3d4": "new" | "viewed" | "dismissed" },
  errorDetails: {
```

```

    "filename.xml": {
      stage, errorCount, warningCount, maxSeverity, topError,
      publicationId, cachedAt
    }
  }
}

```

Exports: - `pruneCache()` — remove entries with `cachedAt > 30` days, and batch IDs with no remaining files - `getBatchStatus(batchId)` → "new" | "viewed" | "dismissed" | undefined - `setBatchStatus(batchId, status)` — load-modify-save cycle - `getCachedError(filename)` → detail object or undefined - `setCachedError(filename, detail)` — load-modify-save cycle - `getDismissedBatches()` → `Set<string>` - `reconcileCache(remoteFileNames: Set)` — remove cache entries for files no longer in S3

Phase 3: Error pipeline module

New file: `src/lib/errorPipeline.js`

Orchestrates S3 listing → filename parsing → grouping → cache reconciliation.
Equivalent of `pipeline.js` for the error review section.

loadErrorBatches(s3, bucketName) — main orchestrator: 1. `pruneCache()` 2. `Promise.all([listAllObjects(..., 'errors/'), listAllObjects(..., 'complete/')])` — parallel 3. For each prefix: strip prefix, filter to `.xml` files only (skip `.xml.json`), run through `parseFilename()`, skip non-matching or `> 30` days old 4. `reconcileCache(allRemoteFileNames)` 5. Group by `batchId` into `Map<batchId, { errors: [], complete: [] }>` 6. Filter out dismissed batches 7. Build batch summary objects: `{ batchId, timestamp, status, errorCount, okCount, errorFiles, completeFiles }` 8. Sort newest first (by most recent file's `LastModified`) 9. Return array

loadErrorDetail(s3, bucketName, filename) — lazy detail fetcher: 1. Check `getCachedError(filename)` — return if cached 2. Download `errors/{filename}.json` via `downloadObject()` 3. Parse JSON, extract: stage, error/warning counts, max severity, top error message 4. Download `errors/{filename}` (the XML itself) via `downloadObject()` 5. Parse XML with `DOMParser`, extract `publicationId` from `<Publication publicationId="...">` 6. Build detail object, call `setCachedError()`, return it

Both JSON and XML downloads happen in parallel per file. Failures are non-fatal per file (returns partial data with `null` for missing fields).

Phase 4: Stores

`src/stores.js`

Add at the bottom: - `errorReviewView` — writable 'list' (list view vs detail view, persists across tab switches) - `errorReviewBatchId` — writable `null` (which batch is currently being viewed)

These are stores rather than component-local state so the user's position is preserved if they accidentally click the Upload tab and back.

Phase 5: Components

`src/components/ErrorReview.svelte` — rewrite (section root)

Manages: credentials check, S3 loading, view routing (list vs detail), batch data.

- `$state`: loading, error, batches[]
- `$derived`: hasCredentials (from `$credentials` store), selectedBatch (from batches + `$errorReviewBatchId`)
- `$effect`: auto-load on first mount when credentials are present
- `refresh()`: calls `loadErrorBatches()`, catches errors
- `viewBatch(id)`: sets `errorReviewBatchId`, sets `errorReviewView` to 'detail'
- `backToList()`: resets view stores, re-runs `refresh()` to pick up any dismissals

Template: no-credentials card (with Settings button) | `ErrorBatchList` | `ErrorBatchDetail` — depending on state.

Wraps in `<div class="page"><div class="content">` (matching current placeholder pattern and how `App.svelte` renders it without its own wrapper).

New: `src/components/ErrorBatchList.svelte` — batch list

Props: loading, error, batches, onrefresh, onselect, ondismiss.

Card with header, Refresh button, then list of `ErrorBatchCard` components. Empty state: checkmark + “No error reports found.” Loading state: “Scanning S3 bucket...”

New: `src/components/ErrorBatchCard.svelte` — one batch row

Props: batch, onselect, ondismiss.

Clickable card showing: batch ID (short), timestamp, error count badge (red), OK count badge (green), status indicator (NEW badge if unseen). Dismiss button (small, secondary) on the right. If batch has no errors (all OK), show a green “all OK” style instead of making it clickable to drill in.

New: `src/components/ErrorBatchDetail.svelte` — single batch detail

Props: batch, onback.

On mount: marks batch as “viewed” via `setBatchStatus()`. Lazily loads error details for all error files (using `loadErrorDetail()`).

State: `errorDetails`: Map, `loadingFiles`: Set, `fileErrors`: Map.

Shows: - Header with batch ID, timestamp, error/OK counts - “Failed Files” section with `ErrorFileRow` for each error file - “Successful Files” summary (just a count — no need to list individual files) - Bottom button row: Back + Dismiss Batch

File download: `downloadObject()` → create Blob → trigger browser download via temporary `<a>` element.

New: `src/components/ErrorFileRow.svelte` — one error file (expandable)

Props: file, detail, loading, error, ondownloadxml, ondownloadjson.

Uses `<details>` element matching the pattern from `ValidationReport.svelte` (expandable rows with custom triangle, scoped styles).

Summary line: filename (mono), severity badge, stage badge, error count.

Expanded body: - Top error message - Full error table (code | message | severity) from `detail.fullReport.errors[]` - Warnings table if any - Metrics summary line - Download XML / Download JSON buttons

Phase 6: CSS

src/app.css — additions at bottom

New global styles for error review components: - .batch-card — full-width clickable card (flex, border, hover effect matching existing patterns) - .batch-card: hover — highlight with --primary border - .batch-header, .batch-stats, .batch-list — layout helpers

Error file row styles will be **scoped** inside ErrorFileRow.svelte (matching the ValidationReport.svelte pattern with .invalid-row / .invalid-summary / .invalid-body).

Phase 7: Optional — notification badge on Error Review tab

src/App.svelte

After ErrorReview loads batches, update an errorBatchNewCount store. Display as a small red badge on the “Error Review” nav tab when > 0. This is a polish item and can be deferred.

File summary

File	Action
src/lib/s3.js	Modify — batch ID, new S3 ops
src/steps/UploadProgress.svelte	Modify — wire up batch ID
src/lib/errorCache.js	Create — localStorage cache
src/lib/errorPipeline.js	Create — S3 listing + grouping
src/stores.js	Modify — add 2 error review stores
src/app.css	Modify — batch card styles
src/components/ErrorReview.svelte	Rewrite
src/components/ErrorBatchList.svelte	Create
src/components/ErrorBatchCard.svelte	Create
src/components/ErrorBatchDetail.svelte	Create
src/components/ErrorFileRow.svelte	Create
src/App.svelte	Modify (optional — badge)

Implementation order: s3.js → UploadProgress → errorCache.js → errorPipeline.js → stores.js → app.css → ErrorFileRow → ErrorBatchCard → ErrorBatchList → ErrorBatchDetail → ErrorReview → App.svelte

Verification

1. npm run build — must succeed with no errors
2. Manual test: upload a few rows with ?debug=true to verify batch ID appears in S3 keys
3. Manually place test .xml and .xml.json files in errors/ prefix to verify the error review listing, grouping, detail loading, dismissal, and download work correctly
4. Verify the filename regex correctly accepts new-format filenames and rejects old-format ones
5. Check localStorage tep_error_cache is populated after viewing a batch and survives page reload
6. Test edge cases: no credentials, empty bucket, S3 errors, corrupt localStorage